

CI/CD Pipeline Guide — Family Music Scrobbler PWA

CI/CD Pipeline Guide - Family Music Scrobbler PWA

A complete reference for the Continuous Integration and Continuous Deployment pipeline used to build, test, and deliver the Family Music Scrobbler PWA. This guide covers GitHub Actions workflows, Docker image publishing, automated testing, and production deployment to the Intel NUC.

1. Overview

The CI/CD pipeline ensures:

- Automated testing on every commit
- Docker images built and published to GitHub Container Registry (GHCR)
- Backend and frontend validated before deployment
- Production updates triggered safely and predictably

The pipeline runs on **GitHub Actions** and deploys to the **Intel NUC (Fedora Server)**.

2. Pipeline Structure

The pipeline consists of three major workflows:

1. CI - Build & Test

Triggered on:

- push
- pull_request

Runs:

- Backend unit tests
- Frontend unit tests
- Linting
- Type checks

2. CD - Build & Publish Docker Images

Triggered on:

- push to main
- Release tags

Builds:

- Backend multi-stage Docker image
- Frontend production build image

Publishes:

- ghcr.io/notascooby25/family-scrobber-backend:<sha>
- ghcr.io/notascooby25/family-scrobber-frontend:<sha>

3. Deployment - Pull & Restart on NUC

Triggered manually or via webhook.

Runs on NUC:

- docker compose pull
- docker compose up -d



3. CI Workflow (Build & Test)

Backend Tests

- Run FastAPI tests via pytest
- Validate database models
- Validate ingestion worker logic

Frontend Tests

- Run React tests via vitest
- Validate components
- Validate hooks

Static Analysis

- flake8 for Python linting
- eslint for JavaScript linting
- mypy for type checking

Example Steps

```
pytest backend/app/tests
npm run test --prefix frontend
flake8 backend
eslint frontend/src
```

4. CD Workflow (Build & Publish Docker Images)

Multi-Stage Backend Build

Stages:

1. Builder (install dependencies)
2. Runtime (copy app + dependencies)

Frontend Build

Stages:

1. Node builder (React build)
2. NGINX runtime (serve static files)

Image Tagging Strategy

- latest for main branch
- <commit-sha> for reproducibility

Publishing to GHCR

Images pushed to:

```
ghcr.io/notascooby25/family-scrobber-backend  
ghcr.io/notascooby25/family-scrobber-frontend
```

5. Secrets & Environment Variables

GitHub Secrets

- GHCR_TOKEN - for pushing images
- SPOTIFY_CLIENT_ID
- SPOTIFY_CLIENT_SECRET
- AES_KEY
- VAPID_PRIVATE_KEY
- VAPID_PUBLIC_KEY

Production Secrets (NUC)

Stored in:

```
/opt/family-scrobber/secrets/
```

6. Deployment Workflow (Intel NUC)

Step 1 - Pull Latest Images

```
docker compose -f infra/docker-compose.yml pull
```

Step 2 - Restart Stack

```
docker compose -f infra/docker-compose.yml up -d
```

Step 3 - Run Database Migrations

```
docker exec backend alembic upgrade head
```

Step 4 - Verify Health

```
curl http://localhost:8000/healthz
```

7. Webhook-Based Auto-Deployment (Optional)

You may configure a webhook on GitHub to notify the NUC when new images are published.

NUC receives webhook → runs:

```
docker compose pull  
docker compose up -d
```

8. Testing Strategy in CI

Backend

- Unit tests for ingestion worker
- API endpoint tests
- Database model tests

Frontend

- Component rendering tests
- Hook behaviour tests
- PWA service worker tests

Integration

- API + frontend integration tests
-

9. Monitoring CI/CD Health

GitHub Actions Dashboard

Check:

- Build failures
- Test failures
- Docker push failures

NUC Logs

```
docker logs backend
docker logs frontend
```

Deployment Status

```
docker ps
curl http://localhost:8000/healthz
```

10. Troubleshooting

Build Failures

- Check dependency versions
- Check Dockerfile syntax
- Check missing environment variables

Test Failures

- Review failing test output
- Check API contract changes

Deployment Failures

- Check NUC disk space
 - Check Docker service status
 - Check secrets mounting
-

11. Summary

This CI/CD pipeline ensures:

- Reliable automated testing

- Reproducible Docker builds
- Safe production deployment
- Continuous delivery of new features

It provides a robust foundation for long-term maintainability and home-lab deployment.